

Relatório — Laboratório de Inspeção HTTP/HTTPS — Fluxo B (sem privilégio administrativo)

Como usar este template: substitua os campos [...] pelas suas respostas, anexe as capturas de tela na pasta evidencias/ e referencie-as onde indicado. Preserve a formatação markdown (tabelas, blocos de código) para facilitar a correção.

Observação: toda a análise prática deste relatório é feita sobre tráfego HTTP em texto claro. A análise de HTTPS é teórica, baseada na fundamentação do readme.md do repositório.

Identificação

Campo	Valor
Nome	Isabelle de Oliveira Jorge Fonseca
RA	0050482413031
Disciplina	Redes de Computadores
Turma	ADS - Noturno
Data	12/05/2026
Fluxo	B — Aluno sem privilégio de administrador
SO utilizado	[Windows 11]
Ferramenta de proxy	Fiddler Classic per-user
Navegador(es)	Chrome
HTTPS-First Mode / HTTPS-Only desabilitado?	Sim

Atividade 1 — Primeira captura (http://example.com)

Captura de tela: evidencias/atv1_sessao.png

Request-line enviada:

GET <http://example.com/> HTTP/1.1 Host: example.com Connection: keep-alive Upgrade-Insecure-Requests: 1 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,;q=0.8,application/signed-exchange;v=b3;q=0.7 Accept-Encoding: gzip, deflate Accept-Language: pt-BR,pt-PT;q=0.9,pt;q=0.8,en-US;q=0.7,en;q=0.6

Status-line recebida:

HTTP/1.1 200 OK Date: Tue, 12 May 2026 23:59:01 GMT Content-Type: text/html Transfer-Encoding: chunked Connection: keep-alive Server: cloudflare Last-Modified: Sat, 09 May 2026 09:00:11 GMT Allow: GET, HEAD cf-cache-status: HIT Age: 3942 Content-Encoding: gzip CF-RAY: 9fad5ff14c36e2b1-GIG

173 eQ n 0 +X j M (r ĩ Rŋ wvf m # V3 ?p E '8 j æX/S \ .xF Mq! ccp =8

*** FIDDLER: RawDisplay truncated at 128 characters. Right-click to disable truncation. ***`

Pergunta 1.1

Quantos cabeçalhos o navegador enviou no request? Liste-os.

Resposta: 7

Cabeçalhos: Host Connection Upgrade-Insecure-Requests User-Agent Accept Accept-Encoding Accept-Language

Pergunta 1.2

Qual foi o `Content-Length` da resposta? Se ele não apareceu, registre `Transfer-Encoding`, versão do protocolo ou outro indício observado. O corpo retornado é HTML, texto puro, JSON ou binário? Como você descobriu?

Resposta: O Content-Length não apareceu na resposta. No lugar dele apareceu Transfer-Encoding: chunked, indicando que os dados foram enviados em partes. A resposta utilizou o protocolo HTTP/1.1.

Atividade 2 — Anatomia de um GET (<http://httpbin.org/get?...>)

Captura de tela: [evidencias/atv2_raw.png](#)

Request-line completa:

GET <http://httpbin.org/get?aluno=ISABELLE&curso=redes> HTTP/1.1 Host: httpbin.org Connection: keep-alive Upgrade-Insecure-Requests: 1 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,/;q=0.8,application/signed-exchange;v=b3;q=0.7 Accept-Encoding: gzip, deflate Accept-Language: pt-BR,pt-PT;q=0.9,pt;q=0.8,en-US;q=0.7,en;q=0.6

Cabeçalhos-chave capturados:

Cabeçalho	Valor
Host	httpbin.org
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,/;q=0.8,application/signed-exchange;v=b3;q=0.7

Campos do JSON de resposta:

```
{ "args": { "aluno": "ISABELLE", "curso": "redes" }, "headers": { "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,/;q=0.8,application/signed-exchange;v=b3;q=0.7", "Accept-Encoding": "gzip, deflate", "Accept-Language": "pt-BR,pt-PT;q=0.9,pt;q=0.8,en-US;q=0.7,en;q=0.6", "Host": "httpbin.org", "Upgrade-Insecure-Requests": "1", "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36", "X-Amzn-Trace-Id": "Root=1-6a03c10f-5d3d3b704a6df9a0521cb1df" }, "origin": "170.247.37.68", "url": "http://httpbin.org/get?aluno=ISABELLE&curso=redes" }
```

Pergunta 2.1

O valor do campo `origin` corresponde a qual elemento da rede? Por que normalmente não é o IP local?

Resposta: O campo `origin` corresponde ao IP público da conexão que fez a requisição para o servidor. Normalmente ele não é o IP local porque a rede usa NAT, então o roteador troca o IP privado do computador pelo IP público da internet antes de enviar os pacotes.

Pergunta 2.2

Compare o `User-Agent` enviado com o que aparece no JSON da resposta. Coincidem?

Resposta: Sim. O `User-Agent` enviado no request é o mesmo que aparece no JSON da resposta, mostrando que o servidor recebeu corretamente as informações do navegador utilizado.

Pergunta 2.3

Em `http://httpbin.org/headers`, liste até três cabeçalhos que o servidor vê mas **não aparecem** no Raw do request. De onde vêm? Se não encontrar três, explique por que o resultado pode variar.

Resposta:

Cabeçalho visto pelo servidor	Origem provável	Observação
<code>X-Amzn-Trace-Id</code>	Infraestrutura da Amazon/AWS	Adicionado pelo serviço intermediário da hospedagem
<code>Via</code>	Proxy ou gateway intermediário	Pode ser inserido durante o roteamento
<code>X-Forwarded-For</code>	Proxy/rede intermediária	Pode informar o IP original do cliente

Atividade 3 — POST e envio de formulário (`http://httpbin.org/forms/post` → `/post`)

Captura de tela: `evidencias/atv3_post_raw.png`

Request-line do POST:

POST <http://httpbin.org/post> HTTP/1.1 Host: httpbin.org Connection: keep-alive Content-Length: 175
Cache-Control: max-age=0 Origin: <http://httpbin.org> Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,./;q=0.8,a
pplication/signed-exchange;v=b3;q=0.7 Referer: <http://httpbin.org/forms/post> Accept-Encoding: gzip,
deflate Accept-Language: pt-BR,pt-PT;q=0.9,pt;q=0.8,en-US;q=0.7,en;q=0.6

custname=Isabelle+Fonseca&custtel=13996768565&custemail=isabellefonseca%40gmail.com&size=large&topping=bacon&topping=cheese&topping=onion&delivery=21%3A00&comments=capriche%21

Cabeçalhos do request:

Cabeçalho	Valor
<code>Content-Type</code>	application/x-www-form-urlencoded
<code>Content-Length</code>	175

Corpo completo do request:

```
custname=Isabelle+Fonseca&custtel=13996768565&custemail=isabellefonseca%40gmail.com&size=large&topping=bacon&topping=cheese&topping=onion&delivery=21%3A00&comments=capriche%21
```

Trecho do JSON de resposta (campo `form`):

```
"form": { "comments": "capriche!", "custemail": "isabellefonseca@gmail.com", "custname": "Isabelle Fonseca", "custtel": "13996768565", "delivery": "21:00", "size": "large", "topping": [ "bacon", "cheese", "onion" ] }
```

Pergunta 3.1

Qual o formato do corpo? Como esse formato codifica caracteres especiais (espaço, acentos)?

Resposta: O corpo está no formato `application/x-www-form-urlencoded`. Nesse formato os dados são enviados como pares `campo=valor` separados por `&`.

Pergunta 3.2

Comparando **Request** → **WebForms** e **Request** → **Raw**: qual das duas corresponde literalmente aos bytes enviados no socket TCP?

Resposta: A aba Raw corresponde literalmente aos bytes enviados no socket TCP, porque ela mostra a requisição exatamente como foi transmitida pela rede. A aba WebForms apenas organiza os dados de forma mais legível.

Pergunta 3.3 — Composer

Envie manualmente via Composer um `POST` para `http://httpbin.org/post` com JSON. Registre a resposta. Qual campo do JSON confirma que o servidor interpretou o JSON?

Captura de tela: `evidencias/atv3_composer.png`

Response JSON (trecho relevante):

```
{ "json": { "protocolo": "HTTP", "versao": "1.1" } }
```

Resposta: O campo `"json"` confirma que o servidor interpretou corretamente o conteúdo enviado como JSON, porque os dados enviados aparecem organizados dentro dele.

Atividade 4 — Catálogo de status codes (`http://httpbin.org/...`)

Captura de tela (lista do Fiddler com as 7 sessões): `evidencias/atv4_lista.png`

#	Método	URL	Status-line	Body?
2	GET	/status/200	HTTP/1.1 200 OK	Sim
3	GET	/redirect-to?...	HTTP/1.1 301 Moved Permanently	Não
4	GET	/status/404	HTTP/1.1 404 NOT FOUND	Sim
5	GET	/status/418	HTTP/1.1 418 I'M A TEAPOT	Sim
6	GET	/status/500	HTTP/1.1 500 INTERNAL SERVER ERROR	Sim
7	GET	/status/503	HTTP/1.1 503 SERVICE UNAVAILABLE	Sim
8	GET	/cache	HTTP/1.1 304 NOT MODIFIED	Não

Pergunta 4.1

Em qual dos status o corpo está ausente/tamanho zero? Isso é obrigatório pela especificação ou depende do servidor?

Resposta: Nos meus testes, os status 301 Moved Permanently e 304 Not Modified vieram sem body/tamanho zero. No caso do 304, isso é esperado pela especificação, já que ele apenas informa que o navegador pode continuar usando o conteúdo armazenado em cache. Já no 301, o servidor pode ou não enviar um corpo na resposta, então isso depende da implementação do servidor.

Pergunta 4.2

No `301`, qual cabeçalho da resposta informa para onde ir? O que aconteceria se estivesse ausente?

Resposta: O cabeçalho responsável pelo redirecionamento é o Location. Ele informa para qual endereço o navegador deve ir após receber o status 301. Se esse cabeçalho estivesse ausente, o navegador não saberia qual destino acessar.

Pergunta 4.3

Diferença semântica entre `200`, `304` e `404` do ponto de vista do cache do navegador.

Resposta: O status 200 OK indica que o recurso foi encontrado normalmente e o conteúdo é enviado pelo servidor. O 304 Not Modified informa que o recurso não foi alterado desde a última requisição, permitindo que o navegador use a versão salva em cache. Já o 404 Not Found indica que o recurso não existe ou não foi encontrado no servidor.

Atividade 5 — Identificação de cabeçalhos

(`http://httpbin.org/response-headers?...` + `/gzip`)

Captura de tela (Inspectors → Headers): `evidencias/atv5_headers.png`

Cabeçalho	Req/Resp	Valor capturado	Função em uma frase
<code>Host</code>	Request	<code>httpbin.org</code>	Identifica o servidor de destino da requisição.
<code>User-Agent</code>	Request	<code>Mozilla/5.0 (Windows NT 10.0; Win64; x64)...</code>	Informa ao servidor qual navegador e sistema operacional estão sendo usados.
<code>Accept</code>	Request	<code>text/html,application/xhtml+xml,application/xml...</code>	Indica os tipos de conteúdo que o cliente aceita receber.
<code>Accept-Encoding</code>	Request	<code>gzip, deflate</code>	Informa os formatos de compressão aceitos pelo navegador.
<code>Cookie</code>	Request	<code>teste=1</code>	Envia cookies armazenados anteriormente pelo navegador.
<code>Server</code>	Response	<code>unicorn/19.9.0</code>	Identifica o software do servidor utilizado.

Cabeçalho	Req/Resp	Valor capturado	Função em uma frase
Content-Type	Response	application/json	Define o tipo do conteúdo retornado pelo servidor.
Content-Encoding	Response	gzip	Indica que o conteúdo foi compactado antes do envio.
Set-Cookie	Response	teste=1	Envia um cookie para ser armazenado pelo navegador.
Cache-Control	Response	max-age=3600	Define regras de cache para a resposta.
Strict-Transport-Security	—	Não observado	Não aparece em HTTP

Pergunta 5.1

Content-Encoding: gzip / br apareceu? Compare Content-Length, quando presente, com o conteúdo visível. O que explica a diferença?

Resposta: Sim, apareceu Content-Encoding: gzip. O tamanho mostrado em Content-Length pode ser menor que o conteúdo visível porque os dados foram compactados antes de serem enviados. Quando o navegador ou o Fiddler descompacta o conteúdo, ele fica maior e legível.

Pergunta 5.2

Cliente envia Accept: application/json mas o recurso só existe em text/html. Qual status code esperar?

Resposta: O status esperado é 406 Not Acceptable, pois o servidor não consegue retornar o formato solicitado pelo cliente.

Pergunta 5.3

`Strict-Transport-Security` apareceu nas respostas HTTP? Por que esse cabeçalho está ausente neste fluxo? (Consulte a RFC 6797.) Qual é seu papel contra downgrades para HTTP puro?

Resposta: O cabeçalho `Strict-Transport-Security` não apareceu nas respostas HTTP observadas. Isso acontece porque o HSTS só é válido em conexões HTTPS, conforme definido pela RFC 6797. Seu papel é fazer o navegador forçar conexões HTTPS em acessos futuros, evitando downgrades para HTTP puro e aumentando a segurança da comunicação.

Atividade 6 — HTTP vs HTTPS (análise sem decifração)

Captura de tela HTTP (`neverss1.com`): `evidencias/atv6_http.png` Captura de tela HTTPS (`https://httpbin.org/get` , apenas `CONNECT`): `evidencias/atv6_https.png`

Pergunta 6.1

Que método HTTP aparece na sessão do `https://httpbin.org/get` ? O que ele faz e por que existe?

Resposta: Na sessão HTTPS apareceu o método `CONNECT`. Esse método é usado para criar um túnel entre o navegador e o servidor HTTPS através do proxy. Depois que o túnel é estabelecido, os dados passam criptografados e o Fiddler não consegue ler o conteúdo sem a decifração HTTPS habilitada.

Pergunta 6.2

Tabela comparativa dos campos visíveis ao Fiddler em cada caso:

Campo	Visível em HTTP?	Visível em HTTPS (sem decifração)?
Método	Sim	Parcialmente (<code>CONNECT</code>)
URL completa (path + query)	Sim	Não
Cabeçalhos de request	Sim	Não
Corpo de request	Sim	Não
Status code	Sim	Não
Cabeçalhos de response	Sim	Não
Corpo de response	Sim	Não
Host (via SNI, no <code>CONNECT</code>)	Sim	Sim
IP e porta de destino	Sim	Sim

Pergunta 6.3 (teórica)

O que você veria no Fiddler se tivesse privilégio de administrador e pudesse habilitar *Decrypt HTTPS traffic*? Indique telas/abas e justifique por que essa inspeção exige a instalação de um certificado raiz.

Resposta: Se a opção Decrypt HTTPS traffic estivesse habilitada, o Fiddler conseguiria descriptografar o tráfego HTTPS e mostrar as mensagens completas nas abas Inspectors → Raw, Headers, JSON e TextView. Seria possível visualizar request-line, status-line, cabeçalhos, cookies, parâmetros, JSON e corpo das respostas normalmente, como acontece no HTTP puro. Isso exige a instalação de um certificado raiz no sistema para que o navegador confie no Fiddler como intermediário da conexão HTTPS.

Pergunta 6.4

Por que a técnica de deciptação dos *debugging proxies* não funcionaria contra um usuário se um atacante a tentasse sem instalar o certificado?

Resposta: Sem instalar o certificado raiz, o navegador não confiaria no proxy intermediário e exibiria erros de segurança HTTPS. Isso acontece porque o certificado apresentado pelo proxy não seria reconhecido como válido. A deciptação só funciona quando existe cooperação do usuário, permitindo que o sistema confie no certificado instalado pelo Fiddler.

Atividade 7 — Cookies e sessão (<http://httpbin.org/cookies/...>)

Captura de tela da sequência: [evidencias/atv7_cookies.png](#)

#	URL	Set-Cookie recebido	Cookie enviado
1	/cookies/set?...	disciplina=redes; Path=/ , professor=claudio; Path=/	nenhum
2	/cookies (1ª visita)	nenhum	disciplina=redes; professor=claudio
3	/cookies (reload 1)	nenhum	disciplina=redes; professor=claudio
4	/cookies (reload 2)	nenhum	disciplina=redes; professor=claudio

Pergunta 7.1

Set-Cookie aparece uma vez ou em toda requisição? Justifique.

Resposta: O cabeçalho Set-Cookie aparece apenas na resposta em que o servidor cria ou altera um cookie. Depois disso, o navegador passa a enviar o valor armazenado no cabeçalho Cookie nas próximas requisições.

Pergunta 7.2

Que atributos o Set-Cookie trouxe? Explique cada um presente. Para atributos não observados, registre não observado.

Nota: o httpbin define cookies mínimos — apenas o atributo Path=/ estará presente. Para cada atributo ausente, registre não observado e explique o comportamento padrão do navegador na sua ausência (ex.: sem Expires / Max-Age → cookie de sessão; sem Secure → pode ser enviado por HTTP; sem SameSite → o navegador aplica a política padrão da versão em uso).

Resposta: No teste realizado, apenas o atributo Path=/ apareceu no Set-Cookie. Ele define em quais caminhos do site o cookie pode ser enviado.

Os demais atributos não foram observados:

Domain: define para quais domínios o cookie é válido. Expires e Max-Age: definem o tempo de duração do cookie. Secure: faz o cookie ser enviado apenas em HTTPS. HttpOnly: impede acesso ao cookie via JavaScript. SameSite: controla o envio do cookie entre sites diferentes.

Atributo	Valor	Função	Observado?
Path	/	Define em quais caminhos do site o cookie pode ser enviado	Sim
Domain	—	Define para quais domínios o cookie é válido	não observado
Expires	—	Define a data de expiração do cookie	não observado
Max-Age	—	Define por quantos segundos o cookie permanece válido	não observado
Secure	—	Faz o cookie ser enviado apenas em conexões HTTPS	não observado
HttpOnly	—	Impede acesso ao cookie via JavaScript	não observado
SameSite	—	Controla o envio do cookie entre sites diferentes	não observado

Pergunta 7.3

O atributo `Secure` pode aparecer num cookie recebido por HTTP puro? Qual seria o comportamento esperado? Relacione com o fato de que todo o tráfego desta atividade é visível em texto claro.

Resposta: Sim. O servidor pode enviar um cookie com o atributo `Secure` mesmo em uma resposta HTTP puro. Porém, o navegador não deve enviar esse cookie em conexões HTTP futuras, apenas em HTTPS. Isso existe para evitar que o cookie seja exposto em texto claro na rede. Como nesta atividade o tráfego HTTP é totalmente visível no Fiddler, qualquer pessoa monitorando a conexão conseguiria ler os cookies enviados sem proteção.

Pergunta 7.4

Na aba **Inspectors** → **Cookies**, o cookie armazenado coincide com o campo `cookies` do JSON?

Resposta: Sim. O valor armazenado na aba Cookies coincidiu com o valor retornado no campo `cookies` do JSON da resposta, mostrando que o navegador armazenou corretamente o cookie enviado pelo servidor e o reenviou nas próximas requisições.

Atividade 8 — Manipulação com breakpoints

Esta atividade usa os breakpoints interativos do Fiddler Classic.

Captura de tela da edição do User-Agent: `evidencias/atv8_ua_edit.png`

JSON de resposta após edição:

```
{
  "user-agent": "LaboratorioRedes/1.0 (Aluno ISABELLE)"
}
```

Pergunta 8.1

O servidor pode detectar que o `User-Agent` foi forjado? Discuta.

Resposta: Não de forma totalmente confiável. O servidor recebe apenas o valor enviado no cabeçalho HTTP e normalmente confia nele. Porém, ele pode suspeitar de falsificação caso o comportamento do navegador não combine com o User-Agent informado, como diferenças em recursos suportados, tamanho da tela, JavaScript ou outros cabeçalhos enviados.

Pergunta 8.2

Após editar a status-line de `200 OK` para `404 Not Found`, o que o navegador exibe? Comente o papel do proxy como MITM.

Captura de tela: evidencias/atv8_status_edit.png

Resposta: Após alterar a status-line para HTTP/1.1 404 Not Found, o navegador passou a tratar a resposta como erro 404, mesmo que o servidor original tivesse enviado 200 OK. Isso mostra que o Fiddler atuou como intermediário entre cliente e servidor, modificando a resposta antes dela chegar ao navegador. O navegador confiou na resposta alterada pelo proxy.

Pergunta 8.3

Confirme que todos os breakpoints foram desabilitados.

- [] Breakpoints desabilitados ao final (Shift+F11)

Atividade 9 — Redirecionamento HTTP → HTTPS

Captura de tela: evidencias/atv9_redir.png

Status-line da resposta a `http://httpbin.org/redirect-to?status_code=301&url=https%3A%2F%2Fhttpbin.org%2Fget` :

```
HTTP/1.1 301 Moved Permanently
```

Cabeçalho `Location` **da resposta:**

```
Location: Location: https://httpbin.org/get
```

Pergunta 9.1

Código de status e cabeçalho que direcionaram o navegador para `https://`.

Resposta: O código de status retornado foi 301 Moved Permanently. O cabeçalho responsável por direcionar o navegador para `https://` foi o Location, que indicou o novo endereço <https://httpbin.org/get>.

Pergunta 9.2

Além do redirecionamento 3xx, qual outro mecanismo/cabeçalho faz o navegador passar a forçar HTTPS em visitas futuras? Cite a RFC.

Resposta: Além do redirecionamento 3xx, o mecanismo que faz o navegador forçar HTTPS em acessos futuros é o HSTS (HTTP Strict Transport Security). Ele utiliza o cabeçalho de response Strict-Transport-Security, definido pela RFC 6797. Esse mecanismo instrui o navegador a acessar o domínio somente via HTTPS durante um período determinado.

Pergunta 9.3

Se esse cabeçalho fosse enviado por uma resposta servida via HTTP puro, o navegador deveria obedecer? Justifique com base na RFC.

Resposta: Não. O navegador não deve obedecer ao cabeçalho Strict-Transport-Security quando ele é recebido por HTTP puro, sem TLS. Segundo a RFC 6797, o HSTS só é válido em conexões HTTPS autenticadas. Isso acontece porque um atacante poderia interceptar ou modificar uma resposta HTTP e inserir informações falsas. Confiar em uma política de segurança recebida por um canal inseguro seria contraditório e permitiria ataques de downgrade para HTTP.

7. Impacto prático de `Cache-Control: no-store`.

O Cache-Control: no-store impede que o navegador ou proxies armazenem a resposta em cache. Na prática, isso aumenta a privacidade e segurança, principalmente em páginas com dados sensíveis, como login, banco ou informações pessoais.

8. Como um debugging proxy decifra HTTPS sem violar a criptografia, e por que isso exige cooperação do usuário (e por que, justamente, você não pôde executar essa etapa)?

O debugging proxy funciona como intermediário entre navegador e servidor. Para conseguir decifrar o HTTPS, ele instala um certificado raiz confiável na máquina do usuário. Assim, o navegador passa a confiar no proxy e permite que ele descriptografe o tráfego. Isso exige cooperação do usuário porque o certificado precisa ser instalado manualmente. Neste laboratório isso não foi possível porque o Fluxo B não tinha privilégio de administrador para instalar certificados no sistema.

9. Exemplo de cabeçalho de request que o navegador envia automaticamente, sem a página pedir.

Um exemplo é o cabeçalho User-Agent, que o navegador envia automaticamente para identificar o navegador, sistema operacional e versão utilizada.

10. Se fosse automatizar parte da inspeção mantendo o Fiddler como proxy, que abordagem usaria? Por quê?

Eu utilizaria scripts ou ferramentas automatizadas configuradas para usar o Fiddler como proxy. Assim seria possível capturar e analisar várias requisições automaticamente, facilitando testes, depuração e análise de tráfego sem precisar fazer tudo manualmente.

11. (Exclusiva do Fluxo B) Três cabeçalhos de segurança que não aparecem ou não fazem sentido em respostas HTTP puro. Para cada um, o que aconteceria se enviado por um servidor HTTP? (Cite RFC 6797 para HSTS.)

Resposta:

Cabeçalho	Comportamento esperado sobre HTTP	Referência
Strict-Transport-Security	O navegador ignora o cabeçalho em HTTP puro, pois ele só é válido em HTTPS	RFC 6797
Content-Security-Policy	Pode ser enviado em HTTP, mas perde parte da proteção porque o tráfego pode ser alterado por terceiros	CSP Specification
Public-Key-Pins	Não faz sentido em HTTP porque depende de conexão HTTPS autenticada	RFC 7469

Reflexão final (opcional, até 10 linhas)

O que você aprendeu que não conhecia antes deste laboratório? Há algum cabeçalho, código de status ou comportamento que passou a olhar com mais atenção? Alguma dificuldade que recomendaria evitar para a próxima turma?

Apreendi melhor como funciona a comunicação HTTP e como os navegadores enviam e recebem informações pela rede. Também entendi a diferença prática entre HTTP e HTTPS e como os cabeçalhos influenciam segurança, cache e cookies. O que mais chamou atenção foi ver que o proxy consegue modificar respostas antes de chegarem ao navegador. A maior dificuldade foi lidar com redirecionamentos automáticos para HTTPS e encontrar as sessões corretas no Fiddler.

Encerramento — justificativa de segurança (Fluxo B)

Parágrafo: por que a remoção de certificado é dispensável neste fluxo e por que seria obrigatória para o aluno administrador:

Neste fluxo não foi necessário remover certificados porque nenhum certificado raiz foi instalado no sistema, já que o HTTPS não foi descriptografado. No fluxo do aluno administrador, a remoção seria obrigatória porque o Fiddler instala um certificado confiável para interceptar conexões HTTPS. Manter esse certificado instalado poderia gerar riscos de segurança fora do laboratório.

- ☐ HTTPS-First Mode / HTTPS-Only Mode reabilitado no navegador
- ☐ Fiddler fechado (porta de proxy liberada)
- ☐ Configuração de proxy removida do navegador (se aplicável)

Checklist de entrega

- ☐ Todos os campos [...] substituídos
- ☐ Pasta evidencias/ com capturas nomeadas por atividade (incluindo Atv. 9)
- ☐ 11 questões de verificação respondidas
- ☐ Atividade 9 (redirecionamento HTTP→HTTPS) documentada
- ☐ Justificativa de encerramento redigida
- ☐ Arquivo compactado como NOME_RA_LAB_HTTP_FLUX0B.zip
- ☐ Submetido no Microsoft Teams dentro do prazo